

The image displays two screenshots of a Google Colab notebook titled "shivam mane ai BayesClassifier.ipynb". The notebook is running on a Google Cloud Platform instance, as indicated by the URL in the browser's address bar: `colab.research.google.com/drive/1Z_w4bdw9opEqOUhd_rnEdj6n-Q_aT7_#scrollTo=TeMqF7KGlpKp`.

The first screenshot shows the initial code cells. The first cell imports the necessary libraries: `numpy`, `matplotlib.pyplot`, and `pandas`. The second cell imports `sklearn.model_selection` for `train_test_split`. The third cell imports `sklearn.preprocessing` for `StandardScaler`. The fourth cell imports `sklearn.naive_bayes` for `GaussianNB`. The fifth cell performs the training, splitting the `iris.csv` dataset into training and testing sets, scaling the features, and fitting the `GaussianNB` classifier. The sixth cell predicts the results for the test set and prints them.

The second screenshot shows the continuation of the notebook. The seventh cell prints the predicted test results. The eighth cell calculates the model's accuracy using `accuracy_score` and prints it. The ninth cell calculates the confusion matrix using `confusion_matrix` and prints it. The output of the ninth cell shows the predicted test results, the model accuracy (96.66666666666667 %), and the confusion matrix.

```
[1] import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
dataset = pd.read_csv('iris.csv')
X = dataset.iloc[:, [0, 3]].values
y = dataset.iloc[:, -1].values

# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_state = 0)

# Feature Scaling
# Since our dataset containing character variables
# we have to encode it using LabelEncoder
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Training the Naive Bayes model on the Training set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)

# Predicting the Test set results
y_pred = classifier.predict(X_test)
print("Predicted Test Results : ", y_pred)
print("\n"*20)

[2] # Predicting the Test set results
y_pred = classifier.predict(X_test)
print("Predicted Test Results : ", y_pred)
print("\n"*20)

# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
ac = accuracy_score(y_test, y_pred)
print("Model Accuracy : ", ac*100, "%")
print("\n"*20)
cm = confusion_matrix(y_test, y_pred)
print("Model Confusion Matrix : ")
print(cm)

Predicted Test Results : ['virginica' 'versicolor' 'setosa' 'virginica' 'setosa' 'virginica'
'setosa' 'versicolor' 'versicolor' 'versicolor' 'versicolor' 'versicolor'
'versicolor' 'versicolor' 'versicolor' 'setosa' 'versicolor' 'versicolor'
'setosa' 'setosa' 'virginica' 'versicolor' 'setosa' 'setosa' 'virginica'
'setosa' 'setosa' 'versicolor' 'versicolor' 'setosa']

Model Accuracy : 96.66666666666667 %

Model Confusion Matrix :
[[11  0  0]
 [ 0 13  0]
 [ 0  1  5]]

[3] Start coding or generate with AI.

[4] import numpy as np
from sklearn.preprocessing import StandardScaler
```

NAME: SHIVAM MANE

ROL NO: T094

SUBJECT: AI

SHETH L.U.J AND SIR M.V. COLLEGE

The image displays two sequential screenshots of a Google Colab notebook titled "shivam mane ai BayesClassifier.ipynb".

Top Screenshot: Shows the initial code cells. Cell [1] contains a prompt to start coding. Cell [2] imports necessary libraries: `import numpy as np`, `import pandas as pd`, `from sklearn.model_selection import train_test_split`, `from sklearn.naive_bayes import GaussianNB`, and `from sklearn import metrics`. It then prints "Libraries imported". Cell [14] loads the dataset: `pima_df = pd.read_csv("iris.csv")` and prints "Dataset Loaded". Cell [15] initializes predictor variables: `X = pima_df.drop("species", axis=1)` and `y = pima_df["species"]`. Cell [16] transforms the data using `StandardScaler`: `std = StandardScaler()` and `X = std.fit_transform(X)`.

Bottom Screenshot: Shows the execution of the transformation. Cell [15] is the same as above. Cell [16] now includes `print("Transformed Data")` and `print(X)`. The output of `print(X)` is displayed as a large array of standardized data points, such as `[-5.25066772e-02 -1.05056946e+00 1.37235899e-01 1.75297293e-03]`.

NAME: SHIVAM MANE

ROL NO: T094

SUBJECT: AI

SHETH L.U.J AND SIR M.V. COLLEGE

The image displays two screenshots of a Google Colab notebook titled "shivam mane ai BayesClassifier.ipynb". The notebook is running on a Google Cloud Platform instance, as indicated by the URL in the browser's address bar: `colab.research.google.com/drive/1Z_w4bzdW9opEqOUhd_rnEdj6n-Q_aT7_#scrollTo=TeMqF7KGJpKp`.

The first screenshot shows the initial setup of the Gaussian Naive Bayes model. It includes the following code:

```
[ 4.32165405e-01  8.00654259e-01  9.33355755e-01  1.44795564e+00]
[ 6.86617933e-02 -1.24957601e-01  7.62758643e-01  7.98598793e-01]]

[17] #Split the data into training and test sets.
test_size = 0.30 # taking 70:30 training and test set
seed = 7 # Random number seeding for repeatability of the code
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=seed)

#Created an object for GaussianNB.
model = GaussianNB()

#Fitted the data in the model to train it.
model.fit(X_train, y_train)
print(model)

GaussianNB()

[18] # Made predictions on the test set and stored it in a 'predicted' variable
predicted = model.predict(X_test)
print(predicted)

['virginica' 'versicolor' 'setosa' 'versicolor' 'versicolor' 'setosa'
 'virginica' 'versicolor' 'setosa' 'versicolor' 'virginica' 'versicolor'
 'setosa' 'virginica' 'setosa' 'virginica' 'virginica' 'virginica'
 'setosa' 'versicolor' 'virginica' 'versicolor' 'versicolor'
 'virginica' 'virginica' 'versicolor' 'versicolor' 'virginica' 'virginica'
 'virginica' 'versicolor' 'setosa' 'virginica' 'versicolor' 'setosa'
 'setosa' 'setosa' 'setosa' 'virginica' 'virginica' 'versicolor'
 'virginica' 'virginica' 'versicolor']

[19] # Result Diagnosis
```

The second screenshot shows the continuation of the notebook, including the calculation of the model's score and the prediction of probabilities:

```
[19] # Result Diagnosis
from sklearn.metrics import accuracy_score, confusion_matrix
metrics.confusion_matrix(predicted, y_test)

array([[12,  0,  0],
       [ 0, 13,  2],
       [ 0,  3, 15]])

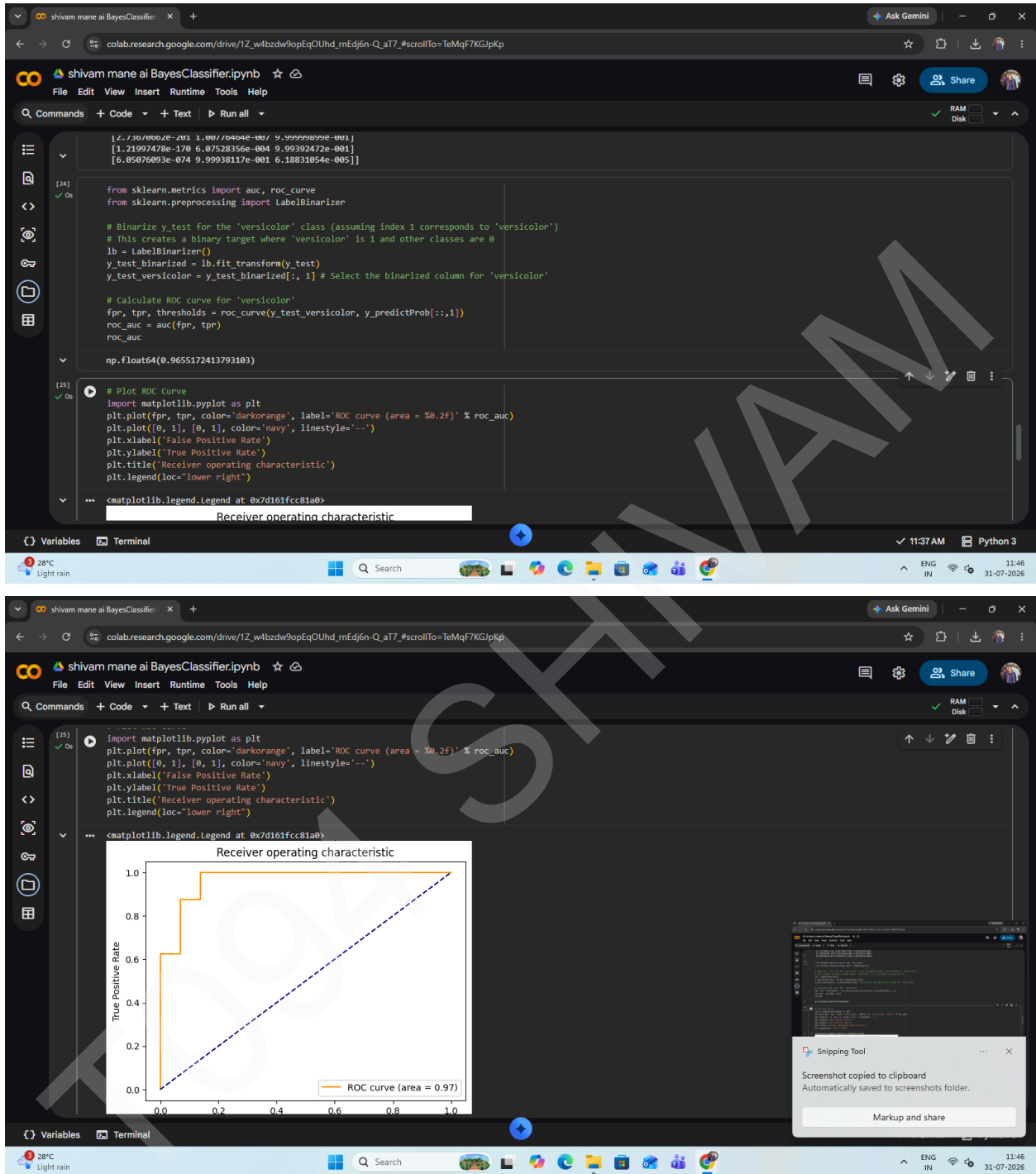
[20] #https://scikit-learn.org/stable/modules/model_evaluation.html
model_score = model.score(X_test, y_test)
model_score

0.8888888888888888

[21] #https://towardsdatascience.com/predict-vs-predict-proba-sikit-learn-bdc45daa5972
#https://scikit-learn.org/stable/search.html?q=predict_proba
y_predictProb = model.predict_proba(X_test)
print(y_predictProb)

[[ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]
 [ 0.92031144  0.07968856  0.00000000]]
```

NAME: SHIVAM MANE
ROL NO: T094
SUBJECT: AI



NAME: SHIVAM MANE

ROL NO: T094

SUBJECT: AI